

No Man's Land:

A Custom SystemVerilog GPU and Game on the DE1-SoC

Rohit Biswas (rb3908) Kambinachi Obioha (kno2117) Nicola Paparella (np2953)

CSEE 4840 — Embedded System Design, Spring 2026

Instructor: Prof. Stephen Edwards

May 12, 2026

Abstract

No Man's Land is a top-down WWI horde-survival game implemented on the Terasic DE1-SoC. The project partitions the system between a custom SystemVerilog GPU peripheral (`nm1_gpu`) running in the Cyclone V FPGA fabric and a C game on the ARM Cortex-A9 hard processor. The GPU drives a 640×480 @ 60 Hz VGA output with tile-mapped background, 32 hardware sprites with per-scanline priority sorting, a 256-entry palette, and an on-chip HUD overlay. The HPS writes the sprite table, palette, tile map, and player-state registers through a 16 KB Avalon-MM lightweight slave and signals a tear-free buffer swap at `VBLANK`. Two build flows exist: a fabric-only smoke test (`hw/quartus`) that confirms the rendering pipeline by initialising the memories from `$readmemh` hex files, and an HPS-integrated Platform Designer system (`nm1_gpu_hw/`) that mounts the peripheral on the lightweight bridge and exposes it to a Linux userspace driver via `/dev/mem`. The smoke-test bitstream is verified on hardware. The HPS-integrated build boots and runs the game loop, but at the time of writing exhibits a banded-stripe rendering artefact and a broken Game-Over screen that are still under debugging. This report covers the architecture, register map, build flow, current state, and open issues.

Contents

1	Introduction	4
2	System Architecture	4
3	Hardware Design	5
3.1	Pixel clock generation (<code>pll_25mhz.v</code>)	5
3.2	VGA timing (<code>vga_timing.sv</code>)	6
3.3	Sprite engine	6
3.3.1	<code>sprite_eval.sv</code>	6
3.3.2	<code>sprite_fetch.sv</code>	6
3.3.3	Double line buffer	7
3.4	Compositor (<code>compositor.sv</code>)	7
3.5	Memories	8
3.6	Avalon-MM slave interface (<code>avalon_slave_iface.sv</code>)	8
3.7	Top-level integration	9
4	Software Design	9
4.1	Game state machine and entity pool (<code>game.c</code> , <code>game.h</code>)	9
4.2	Wave system (<code>wave.c</code> , <code>wave.h</code>)	10
4.3	Auto-attack system (<code>autoatk.c</code> , <code>autoatk.h</code>)	10
4.4	Input subsystem	11
4.5	Rendering pipeline (<code>render.c</code> , <code>render_terminal.c</code>)	11
4.6	Main loop (<code>main.c</code>)	11
5	Hardware–Software Interface	12
5.1	Register map	12
5.2	Frame-commit handshake	13
5.3	Byte-store semantics for the tile map	13
6	Build and Toolchain	13
6.1	Hardware build	13
6.2	ROM generation (<code>hw/gen_rom.py</code>)	14
6.3	Software build	14
6.4	Boot and SD-card image	14

7	Testing and Verification	14
8	Results	14
8.1	Resource utilisation	14
8.2	Timing analysis	15
8.3	Frame timing	15
9	Challenges and Resolutions	16
9.1	Tile-map byte-enable RMW	16
9.2	Single-cycle <code>eval_done</code> against multi-cycle <code>CLEAR_BUF</code>	16
9.3	CDC on <code>ctrl_swap_req</code>	16
9.4	Pixel-clock duty cycle	16
9.5	Active rendering bugs	17
10	Conclusion and Open Work	17
	References	17

1 Introduction

The course’s final-project remit is to design and integrate a custom SystemVerilog peripheral that drives a VGA display and to write a C application that uses the peripheral to do something interesting. We chose a WWI-themed horde-survival game on the principle that it stresses both sides: the FPGA must composite many sprites per scanline at pixel rate, and the CPU must run wave logic, enemy AI, projectile physics, collision checks, an auto-attack system, and a HUD-driven UI within a 16.67 ms frame budget.

The player controls a soldier in a trench at the bottom of the screen. Waves of enemies spawn at the top and march down. The player fires bullets in four directions, picks up ammo drops, and at the end of each wave selects an auto-attack upgrade (mortar, mustard gas, or artillery). The goal is to survive as many waves as possible.

Input is a SNES-style USB gamepad (DragonRise generic, VID 0079:0011) read through Linux evdev. A keyboard fallback was explicitly disallowed by the instructor; we kept a deterministic stub input (`input_fake.c`) for off-line development and demos when the controller is unavailable.

The deliverables and supporting documents are:

- **DESIGN.md** — the original design document with system block diagram, register map, and game design.
- **SETUP.md** — bring-up instructions: host sanity check, JTAG smoke test, and HPS integration on the SD-card image.
- This report and the companion internal deep-dive (`docs/internal_deep_dive.md`).

2 System Architecture

The system is partitioned along the obvious line: anything that runs at pixel rate is in hardware; everything else is in software. The block diagram is shown in Figure 1.

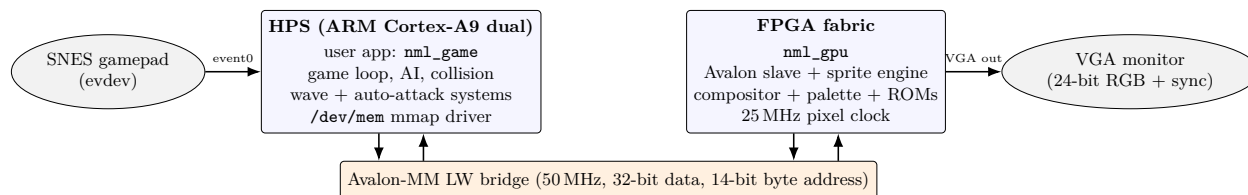


Figure 1: System block diagram. HPS runs the C game; FPGA fabric runs the GPU peripheral; the lightweight bridge carries register writes and reads between them.

The HPS reads input from the USB gamepad, ticks the game state at 60 Hz, writes the resulting sprite table, palette, and tile map into FPGA memory through the Avalon-MM lightweight slave, and signals a frame swap by writing the `CTRL.SWAP` bit. The FPGA latches the new state at the next VSYNC and continues to drive the monitor without ever stalling. A 50 MHz Avalon clock and a 25 MHz pixel clock are the only two clock domains in the design; both derive from the board’s 50 MHz oscillator.

Frame budget at 60 Hz is 16.67 ms. Per VGA frame at 640×480 there are $800 \times 525 = 420,000$ pixel cycles at 25 MHz (16.8 ms wall-clock), of which $640 \times 480 = 307,200$ are visible. The 96-pixel horizontal blanking and multi-line vertical blanking give the sprite engine the time it needs to

prepare each scanline.

3 Hardware Design

The hardware lives in `hw/` and consists of seven SystemVerilog modules plus one Verilog clock divider. A separate Platform Designer project under `nml_gpu_hw/` wraps the same RTL as a Qsys component for HPS integration. Figure 2 shows the module hierarchy.

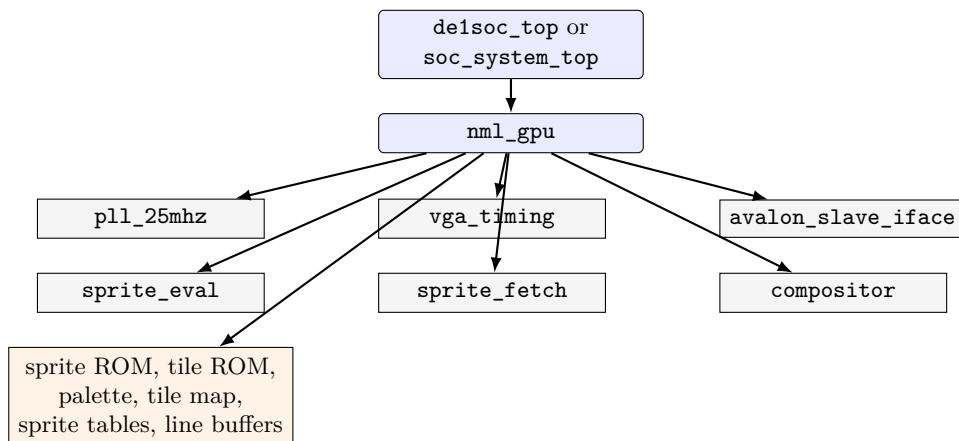


Figure 2: Module hierarchy of the GPU. The same `nml_gpu` is wrapped by `de1soc_top` (Phase 1) or by `soc_system_top` (Phase 2).

3.1 Pixel clock generation (`pll_25mhz.v`)

The VGA standard for 640×480 @ 60 Hz specifies a 25.175 MHz pixel clock. The Cyclone V on-chip PLL cannot produce that exact frequency from the board’s 50 MHz reference, so we use 25 MHz, which monitors in the lab tolerate without issue. For the Phase 1 smoke test, the divider is a single D flip-flop that toggles on every 50 MHz rising edge (`hw/pll_25mhz.v:21-26`). Quartus promotes the divided register output onto a global clock network when it is used as a clock downstream, which is exactly how it is consumed in `hw/nml_gpu.sv:194` and elsewhere.

Listing 1: Divide-by-2 implementation. Replace with `altera_pll` IP in Phase 2.

```

module pll_25mhz (
    input wire refclk,    // 50 MHz reference
    input wire rst,      // active high reset
    output wire outclk_0  // 25 MHz pixel clock
);
    reg clk_div = 1'b0;
    always @(posedge refclk or posedge rst) begin
        if (rst) clk_div <= 1'b0;
        else     clk_div <= ~clk_div;
    end
    assign outclk_0 = clk_div;
endmodule

```

The Phase 2 upgrade path is to replace the divider with an `altera_pll` IP instance. The port names (`refclk`, `rst`, `outclk_0`) already match the IP defaults so swapping in the IP is a drop-in.

The board's VGA DAC (ADV7123) samples R/G/B on its own rising edge. We drive `vga_clk` as the inversion of the internal pixel clock (`hw/nml_gpu.sv:38`) so the DAC samples in the middle of the FPGA's hold window. This avoids hold-time violations into the DAC without adding a register stage.

3.2 VGA timing (`vga_timing.sv`)

`vga_timing` counts a horizontal axis from 0 to 799 and a vertical axis from 0 to 524, producing standard 640×480 @ 60 Hz parameters (`hw/vga_timing.sv:11-21`):

Axis	Active	Front porch	Sync	Back porch	Total
H	640	16	96	48	800
V	480	10	2	33	525

Table 1: VGA timing parameters for 640×480 @ 60 Hz.

`hsync` and `vsync` are active low, matching the VGA spec for this mode. `hblank` is high when `h_count ≥ 640`; `vblank` is high when `v_count ≥ 480`; `visible = !hblank && !vblank`.

3.3 Sprite engine

3.3.1 `sprite_eval.sv`

`sprite_eval` is a small FSM (IDLE → FETCH → EVAL → DONE → IDLE) that walks the 32-entry sprite table once per scanline and selects the top eight sprites by priority that intersect the *next* scanline ($y + 1$). The state machine starts when the top-level asserts `eval_strobe` at $x = 642$, two pixels into HBLANK (`hw/nml_gpu.sv:248`). Each sprite is fetched in FETCH and tested in EVAL; the test is

```
intersects = spr_active &&
             (next_scanline >= spr_y) &&
             (next_scanline < (spr_y + 16));
```

(`hw/sprite_eval.sv:39-41`). All sprites are 16×16. The signed Y compare permits sprites to sit partly above the screen.

The insertion sort is fully unrolled (`hw/sprite_eval.sv:77-94`): each cycle, the incoming sprite is compared against eight registered slots; it lands in the first empty slot or the first slot with lower priority. Slots below the insertion point shift down by one; the lowest-priority slot is dropped if the array is full. Total walk takes 32 cycles for fetch + eval plus one cycle for DONE, well under the ~160-cycle HBLANK budget at 25 MHz.

3.3.2 `sprite_fetch.sv`

`sprite_fetch` is the data-mover. Its FSM (IDLE → CLEAR_BUF → PROCESS_SPRITE → READ_PIXEL → WAIT_PIXEL → WRITE_PIXEL) clears the inactive line buffer to zero (the transparent palette index), then walks the eight sorted sprites from lowest to highest priority, reading 16 palette-index bytes from the sprite ROM per sprite and writing each non-zero, on-screen

pixel into the line buffer (`hw/sprite_fetch.sv:67-131`). The `READ_PIXEL` $\rightarrow$ `WAIT_PIXEL` $\rightarrow$ `WRITE_PIXEL` 3-state inner loop matches the 1-cycle read latency of the sprite ROM (`hw/nml_gpu.sv:194`).

A sprite is addressed by `sprrom_raddr = (spr_id  $\ll$  8) + (pixel_v  $\ll$  4) + actual_u`, where `actual_u = spr_hflip ? (15 - pixel_u) : pixel_u` (`hw/sprite_fetch.sv:107-109`). Vertical flip is in the register layout but not wired in this revision — only `hflip` is honoured.

Walking from lowest to highest priority is a deliberate choice: each sprite’s pixels overwrite whatever was written by a lower-priority sprite, so the highest-priority sprite ends up on top. The buffer-clear fills with zero (transparent) so the compositor can fall through to the tile layer where no sprite wrote.

3.3.3 Double line buffer

Two 640×8 line buffers (`linebuf_A`, `linebuf_B`, `hw/nml_gpu.sv:198-221`) form a ping-pong pair. While `sprite_fetch` fills buffer A during the HBLANK after scanline N , the compositor reads buffer B for scanline $N + 1$; on the next HBLANK, the selector toggles. The toggle fires when `hblank && x == 640` (`hw/nml_gpu.sv:209`), exactly at the transition from visible to blanking.

3.4 Compositor (`compositor.sv`)

The compositor is a 4-stage pipeline that delivers one RGB triplet per pixel. Figure 3 shows the stages.

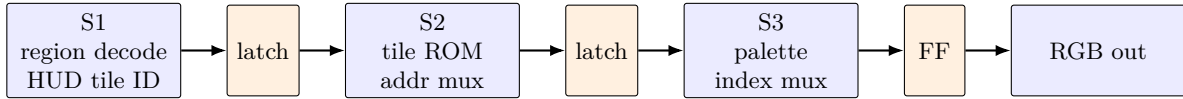


Figure 3: Compositor 4-stage pipeline. Tile ROM and palette RAM each contribute one register cycle.

The stages, with the latches between them, are (`hw/compositor.sv:75-311`):

1. **S1 (combinational):** decode the HUD region predicates (`in_hp_bar`, `in_ammo_label`, `in_ammo_digits`, ..., `in_score_text`), compute the HUD label tile IDs from the local character index, and select the BCD digit nibble from `player_stats`, `hud_aux`, and `score_reg`. Compute the background tile map address from $(x[9 : 3], y[9 : 3])$.
2. **S1 $\rightarrow$ S2 latch** (`hw/compositor.sv:256-268`): register the HUD tile ID, HUD tile X/Y, HP-bar fill flag, sprite pixel from the line buffer, and the background tile map data.
3. **S2 (combinational):** mux the tile ROM read address — either the background path (`tile_id`, `y[2:0]`, `x[2:0]`) or the HUD glyph path (`hud_tile_id`, `hud_tile_y`, `hud_tile_x`).
4. **S2 $\rightarrow$ S3 latch** (`hw/compositor.sv:280-286`): register the HP-bar and HUD-text flags so they line up with the tile ROM read data.
5. **S3 (combinational):** select the palette index — sprite pixel if non-zero, else the tile ROM byte (`hw/compositor.sv:289-292`).

6. **S3→S4 (registered output):** drive `rgb_out`. HUD layer wins in the top 16 pixels; HP bar fills $x \in [0, 191]$ green or dark red; HUD glyph pixels use white on dark grey; everywhere else, the palette RAM output drives the line.

Total latency from x, y to `rgb_out` is three to four cycles (120–160 ns at 25 MHz), well within `VGA_BLANK_N` slop.

3.5 Memories

All six FPGA-side memories are inferred. The relevant ports and depths are summarised in Table 2.

Memory	Storage	Words×Width	Bits	Used for
Sprite table A	[63:0] [0:31]	32×64	2,048	Active sprite list
Sprite table B	[63:0] [0:31]	32×64	2,048	Shadow written by HPS
Palette RAM	[23:0] [0:255]	256×24	6,144	RGB888 colour table
Tile map RAM	[3:0] [7:0] [0:1199]	1200×32	38,400	80×60 tile IDs (packed)
Sprite ROM	[7:0] [0:16383]	16384×8	131,072	64 sprites × 256 B
Tile ROM	[7:0] [0:4095]	4096×8	32,768	64 tiles × 64 B
Line buffer A	[7:0] [0:639]	640×8	5,120	Sprite scanline (ping)
Line buffer B	[7:0] [0:639]	640×8	5,120	Sprite scanline (pong)

Table 2: FPGA-side memory inventory.

The sprite ROM, tile ROM, palette, and shadow sprite table are initialised from `.hex` files via `$readmemh` at synthesis (`hw/nml_gpu.sv:183–192`). The palette and sprite table are subsequently overwritten by the HPS at runtime; the ROMs stay frozen.

Tile-map storage is unusual: the register declaration is `logic [3:0][7:0] tilemap_ram [0:1199]` rather than the more obvious `logic [7:0] tilemap_ram [0:4799]`. The reason is byte-enable handling on the lightweight bridge. ARM emits `STRB` (single-byte stores) for the C driver’s `nml_write_tile()` call, and the bridge translates them into a 32-bit write with `avs_byteenable` selecting one lane. With byte-organised storage, the bridge’s implicit read-modify-write would see only one byte of the word and write back zeros for the other three, silently clobbering 75% of the tile map (`hw/nml_gpu.sv:138–149`). Packed four-lane storage matches Quartus’s M10K byte-enable inference template; each `tilemap_ram[idx][i]` lane updates independently.

3.6 Avalon-MM slave interface (`avalon_slave_iface.sv`)

The peripheral is a 32-bit lightweight Avalon-MM slave with a 14-bit byte-address window (16 KB). The full register map is reproduced in §5.

Reads of the scalar registers (`CTRL`, `STATUS`, `BG_SCROLL`, `IRQ_MASK`, player state, `HUD_AUX`) return combinationally on the read cycle. Reads of the sprite-table, palette, and tile-map regions go through an `always_ff` register on `*_rdata_sw` lines and therefore arrive one cycle late. To make this work without breaking the master, the slave asserts `avs_waitrequest` for the first cycle of any RAM-region read and releases it on the second (`hw/avalon_slave_iface.sv:133–141`).

Writes to the CTRL register specifically handle SWAP by widening the single-cycle Avalon strobe into a 7-cycle counter (`swap_req_cnt`, `hw/avalon_slave_iface.sv:195-209`). The widening matters because the swap signal crosses from the 50 MHz Avalon clock into the 25 MHz pixel clock, where it is captured by a three-flop synchroniser (`hw/nml_gpu.sv:94-106`) and edge-detected to produce a one-pulse event in the pixel-clock domain. A single 20 ns pulse on the Avalon side is too narrow for the 40 ns pixel clock to catch reliably; holding for 140 ns (seven 50 MHz cycles) makes the crossing deterministic.

3.7 Top-level integration

`nml_gpu.sv` is the integration shell. It instantiates the slave, the timing generator, sprite eval, sprite fetch, the compositor, and the PLL, and declares the inferred memories. The active and shadow sprite tables are kept inside this file rather than inside the slave because the swap logic lives here: when `swap_pending` is set and `vsync` is high, the 32-entry active table is bulk-assigned from the shadow on a single clock edge (`hw/nml_gpu.sv:109-118`).

`deisoc_top.sv` is the Phase 1 wrapper. It ties the Avalon inputs off (no master in the standalone build), wires reset to `KEY[0]`, mirrors `VGA_BLANK_N` to `LEDR[1]` as a heartbeat, and passes the VGA outputs through. The Phase 2 wrapper is `nml_gpu_hw/soc_system_top.sv`, which instantiates the Platform Designer system, exposes `nml_gpu`'s slave on the HPS-to-FPGA lightweight bridge, and connects the same VGA pins.

4 Software Design

The software lives in `sw/` and is structured as nine translation units plus three headers. Three build variants share the same sources (Table 3).

Variant	Target	Renderer	Input	Output
<code>make</code> (default)	armhf	<code>render.c</code>	<code>input_real.c</code>	<code>nml_game</code>
<code>make fake-input</code>	armhf	<code>render.c</code>	<code>input_fake.c</code>	<code>nml_game_fake</code>
<code>make terminal</code>	host	<code>render_terminal.c</code>	<code>input_fake.c</code>	<code>nml_game_term</code>
<code>make native</code>	board native	<code>render.c</code>	<code>input_real.c</code>	<code>nml_game</code>

Table 3: Build variants (`sw/Makefile:42-72`).

The terminal build also defines `NML_TERMINAL_BUILD`, which compiles out the FPGA paths in `main.c`.

4.1 Game state machine and entity pool (`game.c`, `game.h`)

State is held in a single struct (`sw/game.h:93-133`). The active states are `STATE_PLAYING`, `STATE_LEVELUP`, and `STATE_GAMEOVER`. A fixed-size pool of 64 entities (`MAX_ENTITIES`, `sw/game.h:6`) holds the player, enemies, bullets, hazards, and ammo drops. Each entity tracks kind, active flag, position, velocity, hp, an animation phase, a TTL, a payload, and a fire cooldown (`sw/game.h:79-90`).

`game_tick()` (`sw/game.c:428-485`) dispatches on state:

- In `STATE_PLAYING`, it updates the player, runs the wave spawner, moves enemies, advances bullets, ages hazards, runs auto-attack timers, and resolves collisions. On wave clear it transitions to `STATE_LEVELUP`. On $HP \leq 0$ it transitions to `STATE_GAMEOVER` and deactivates every entity including the player so the death screen has a clean tilemap to draw into.
- In `STATE_LEVELUP`, the player picks one of three offered weapons with the D-pad and confirms with the B button (`sw/game.c:416-426`). On confirm, the chosen weapon is upgraded and the next wave starts.
- In `STATE_GAMEOVER`, `START` restarts the game by re-calling `game_init()`.

Enemy motion (`sw/game.c:219-254`) is a per-enemy random walk in X combined with vertical pursuit of the player. The walk decision re-rolls every 12 frames (`ENEMY_DIR_FRAMES`) using a per-enemy seed (`phase`) so the cohort does not collapse into a single column where the player's bullet stream would mop them up trivially. Armed enemies also fire bullets back on a per-enemy cooldown jittered at spawn (`sw/game.c:21-23, 239-246`).

Collision resolution is AABB at 16×16 (`sw/game.c:298-312`). All kills go through `apply_damage()` (`sw/game.c:318-333`) so that score, kill counter, and ammo-drop probability are recorded uniformly across weapon paths. The drop probability is 30% per kill (`sw/game.h:144, AMMO_DROP_PERCENT`).

4.2 Wave system (`wave.c`, `wave.h`)

The wave table is a hard-coded array of five entries (`sw/wave.c:9-16`):

Wave	Spawn period (frames)	Total	Armed	Armed HP	Speed
1	60	6	0	1	1
2	50	8	1	2	1
3	45	10	2	2	2
4	35	12	4	2	2
5	25	16	6	3	3

After wave 5 the table clamps to its last entry, so the game can keep running indefinitely at maximum difficulty. `wave_advance()` top-ups ammo (+40, capped at 99), artillery (+2, capped at 5), and gas (+2, capped at 5) on every transition out of `STATE_LEVELUP`.

4.3 Auto-attack system (`autoatk.c`, `autoatk.h`)

Three auto-attack kinds exist (`sw/game.h:60-65`):

- **Mortar** is timer-driven. At level 1 it fires every 180 frames, decreasing by 30 frames per upgrade, floored at 60 frames (`sw/autoatk.c:10-20, 39-45`). Each shell launches upward from the player's column with random horizontal jitter and a TTL of 200 frames.
- **Artillery** is player-triggered with the L shoulder button. On fire it kills every enemy in a 16-pixel column above the player (`sw/autoatk.c:200-219`) and paints a vertical column of `TILE_BEAM` glyphs into the tile map for six frames as the visual effect.
- **Mustard gas** is player-triggered with the R shoulder. It spawns a single `ENT_HAZARD` at the player's position with a TTL of 90 frames. The cloud grows from 16×16 px to 48×32 px over the first 30 frames, holds, and shrinks over the last 15 (`sw/autoatk.c:160-178`).

Each weapon has a per-button input cooldown of 20 frames (`ABILITY_INPUT_COOLDOWN`, `sw/game.h:145`) to debounce noisy SNES shoulder buttons.

4.4 Input subsystem

The input API is a single function: `uint16_t input_read(int frame)` (`sw/input.h:6`). Either `input_real.c` (the DragonRise reader) or `input_fake.c` (the deterministic stub) is linked.

The real reader opens `/dev/input/event0` (the only evdev node that shows up for the gamepad on the DE1-SoC kernel — `joydev` is not built in, so there is no `/dev/input/js0`). It drains pending events on every call, updating an internal button bitmask and axis values (`sw/input_real.c:127-147`). The D-pad is reported as `ABS_X/ABS_Y`; axes are converted to four directional bits with a deadzone equal to a quarter of the reported range, queried via `EVIOCGABS` so the same code works whether the pad reports $-1..1$, $0..255$, or signed 16-bit values.

4.5 Rendering pipeline (`render.c`, `render_terminal.c`)

`render_frame()` (`sw/render.c:342-421`) is called once per tick. It writes the entity pool into the FPGA sprite table:

- Slot 0 is always the player.
- Slots 1–31 hold active non-player entities in pool order.
- Gas hazards expand into up to four sprite slots (`sw/render.c:98-111`).
- Bullets get priority 2 (low), enemies and hazards priority 1, the player priority 0 (high).

Leftover slots are explicitly hidden by `nml_hide_sprite()` so prior frames do not ghost (`sw/render.c:407-409`). The HUD mailbox registers are updated every frame.

The terminal renderer (`sw/render_terminal.c`) mirrors the slot allocation logic exactly but prints each sprite slot to stdout as an ASCII glyph. This lets the game logic be exercised in CI and on a laptop without hardware.

4.6 Main loop (`main.c`)

`main()` (`sw/main.c:171-274`) is a fixed-timestep 60 Hz loop:

1. Read input (`input_read`).
2. Tick the game (`game_tick`).
3. Render (`render_frame` writes the sprite table).
4. Commit (`nml_commit_frame()` sets `CTRL.SWAP`, then polls `STATUS.SWAP_PENDING` until it clears or a 200,000-iteration timeout trips).
5. Sleep the remainder of the 16,666 μ s frame budget.

On the host (terminal build), `nml_commit_frame()` is `#ifdef`-ed out and `nanosleep` is the sole pacing source. On the board, the `SWAP` poll synchronises the loop with `VBLANK`.

Palette initialisation happens once at startup (`init_palette_runtime()`, `sw/main.c:139-168`). The HPS writes sixteen entries covering player, enemies, projectiles, dirt, grass, and the white sprite border. This step shadows the `$readmemh` initialisation; once it runs, the software is the authoritative source of palette data.

5 Hardware–Software Interface

The peripheral occupies 16 KB starting at 0xFF200000 in HPS physical address space (the lightweight bridge base on the DE1-SoC). The userspace driver `mmap()`s the whole window via `/dev/mem` (`sw/nml_gpu.c:33–59`) and exposes typed writers.

5.1 Register map

Offset	Name	W	Acc.	Layout
0x0000	CTRL	32	R/W	[0]=ENABLE, [1]=SWAP (auto-clear), [2]=HUD_ON
0x0004	STATUS	32	R	[0]=VBLANK, [1]=SWAP_PENDING, [15:8]=frame_ctr (wired 0 at present)
0x0008	BG_SCROLL	32	R/W	[15:0]=scroll_x (signed), [31:16]=scroll_y (signed)
0x000C	IRQ_MASK	32	R/W	[0]=VBLANK IRQ enable (Phase 2; wired 0 here)
0x0010	PLAYER_POS	32	R/W	[15:0]=px, [31:16]=py
0x0014	PLAYER_STATS	32	R/W	[7:0]=hp, [15:8]=wave BCD (2 digits), [31:16]=level
0x0018	SCORE	32	R/W	[23:0]=score BCD (6 digits)
0x001C	KILL_COUNT	32	R/W	32-bit kill counter (HUD-only, optional)
0x0020	HUD_AUX	32	R/W	[7:0]=ammo BCD (2 digits), [11:8]=art, [15:12]=gas
0x0100–0x01FF	Sprite table	256 B	R/W	32 × 8 B entries
0x0400–0x07FF	Palette	1 KB	R/W	256 × 4 B; [23:0]=RGB888
0x1000–0x22BF	Tile map	4.7 KB	R/W	80×60 bytes; byte = tile ID

Table 4: Avalon-MM register map.

Each sprite-table entry is two 32-bit words:

Word	Bits	Field
W0	15:0	x (signed)
W0	31:16	y (signed)
W1	7:0	sprite_id
W1	15:8	flags
W1	23:16	palette_off
W1	31:24	reserved

The `flags` byte packs (`sw/nml_gpu.h:58–69`): bit 2 hflip, bit 3 vflip, bits 6:4 priority (0 = highest), bit 7 ACTIVE. The **ACTIVE** bit is the flag that `sprite_eval` actually checks at bit 47 of the 64-bit packed word; the original design used sentinel coordinates exclusively, but the hardware now also honours the explicit active bit (`sw/nml_gpu.h:12–16`, `hw/sprite_eval.sv:32–34`).

5.2 Frame-commit handshake

The frame-commit sequence is:

1. CPU writes the sprite table, palette, tile map, player state, and HUD aux fields. All writes land in shadow registers/RAMs immediately.
2. CPU writes `CTRL.SWAP = 1`. The slave widens the strobe to seven 50 MHz cycles (`hw/avalon_slave_iface.sv:199-209`), so the pixel clock can synchronise the request through three flip-flops and capture an edge (`hw/nml_gpu.sv:94-106`). `SWAP` auto-clears in the same cycle it was written.
3. At the next `VSYNC`, the 32-entry sprite table is bulk-assigned from shadow to active in one clock edge (`hw/nml_gpu.sv:113-118`). `swap_pending` clears.
4. CPU polls `STATUS.SWAP_PENDING` until it falls or the 200,000-iteration timeout trips (`sw/nml_gpu.c:179-190`).

The palette and tile map do not go through the swap handshake. Both have one logical bank that is written in the Avalon clock domain and read in the pixel clock domain. Visible artefacts from mid-frame writes are possible in theory; in practice palette and tile updates are rare, so we left it that way.

5.3 Byte-store semantics for the tile map

`nml_write_tile()` issues a single-byte `STRB` to the tile-map region (`sw/nml_gpu.c:92-99`). The lightweight bridge translates this into a 32-bit Avalon write with `avs_byteenable` set to one nibble. On the slave side, the packed-byte tile-map storage logic `[3:0][7:0] tilemap_ram` exposes four byte-enable lanes per M10K word and writes only the addressed lane (`hw/nml_gpu.sv:158-162`). Reads return the full word; the pixel side selects one of the four bytes per pixel based on `tilemap_raddr[1:0]` (`hw/nml_gpu.sv:169-173`).

This packed layout is non-obvious but is the difference between a correct tile map and one where only every fourth column receives the write.

6 Build and Toolchain

6.1 Hardware build

Quartus Prime Lite 21.1 is the synthesis and place-and-route tool. Two flows exist.

Phase 1 (`hw/quartus/`). A standalone bitstream that places `nml_gpu` inside `de1soc_top` with the Avalon slave tied off. The four `.hex` files generated by `hw/gen_rom.py` are picked up via `$readmemh` at synthesis time, courtesy of `SEARCH_PATH "..."` in the QSF (`hw/quartus/nml_gpu.qsf`). Compilation runs in 63s on a lab machine and produces `output_files/nml_gpu.sof` (for JTAG) and `output_files/nml_gpu.rbf` (raw binary for SD-card boot).

Phase 2 (`nml_gpu_hw/`). A Platform Designer system with an HPS instance, a clock bridge, an AXI-to-Avalon adapter on the HPS-to-FPGA lightweight bridge, and the `nml_gpu` component as an Avalon slave. The component descriptor lives in `nml_gpu_hw.tcl` and sets the device-tree metadata (`compatible = "csee4840,nml_gpu-1.0"`) so the kernel exposes the peripheral at `/proc/device-tree/sopc@0/bridge@0xc0000000/vga@0x100000000/`.

6.2 ROM generation (hw/gen_rom.py)

gen_rom.py is a 777-line Python script that emits four hex files: `sprite_rom.hex` (16 384 bytes), `tile_rom.hex` (4 096 bytes), `palette.hex` (256 RGB888 entries), and `sprite_table.hex` (32 initial sprite slots).

6.3 Software build

The userspace binary cross-compiles with `arm-linux-gnueabihf-gcc` (`sw/Makefile:23`). It links statically (`-static`) so it runs on the board without a matching glibc, and uses standard POSIX APIs (`/dev/mem`, `mmap`, `evdev`, `nanosleep`, `clock_gettime`). No threading; everything is in one process loop.

6.4 Boot and SD-card image

The Phase 2 image follows the standard DE1-SoC Linux boot flow: preloader → U-Boot → kernel (Linux 4.19 from `altera-fpga/linux-socfpga`) → root filesystem on the ext4 partition. The FPGA RBF and DTB sit on the FAT boot partition. U-Boot loads the bitstream into the FPGA fabric before launching the kernel, so the smoke-test scene is already on the VGA monitor by the time userspace comes up. After login, the operator copies `nml_game` onto the board and runs it as root.

7 Testing and Verification

Three test rungs:

- **Phase 0 (host).** `cd sw && make terminal && ./nml_game_term` runs the entire game loop without hardware, printing every frame's sprite table to stdout. The deterministic input stub exercises every weapon path on a fixed cycle.
- **Phase 1 (FPGA only).** With the `.sof` loaded over JTAG, the monitor shows the smoke-test scene (dark battlefield, green player square, two red enemies, a yellow bullet) within one second. This proves the entire pixel pipeline.
- **Phase 2 (HPS-integrated).** With the RBF and DTB on the SD card, the kernel exposes the peripheral, and `./nml_game` mmmaps it. The game loop runs at 60 Hz, the controller drives the player, and the terminal logs wave starts and clears. At the time of writing this configuration boots and the game runs, but the VGA output has a vertical-stripe rendering bug and the Game-Over screen is broken — see §9 and the deep-dive companion for details.

8 Results

8.1 Resource utilisation

From `hw/quartus/output_files/nml_gpu.fit.summary` (Phase 1 build, Cyclone V SE 5CSEMA5F31C6):

Resource	Used	Available	Fraction
ALMs	205	32,070	< 1%
Registers	365	—	—
Pins	54	457	12%
Block memory bits	86,144	4,065,280	2%
M10K blocks	14	397	4%
DSP blocks	0	87	0%
PLLs	0	6	0%

Table 5: Phase 1 resource utilisation.

Total compile time: 63 s (Analysis & Synthesis 13 s, Fitter 30 s, Assembler 12 s, Timing Analyzer 8 s).

8.2 Timing analysis

`hw/quartus/output_files/nml_gpu.sta.summary` reports negative slack on the slow-1100mV-85°C corner:

Corner	Clock	Setup slack	TNS
Slow 1100mV 85°C	<code>clk_div</code>	−5.111 ns	−1409.97 ns
Slow 1100mV 85°C	<code>CLOCK_50</code>	−3.506 ns	−3.506 ns
Fast 1100mV 85°C	<code>clk_div</code>	−2.654 ns	−725.39 ns

All hold checks pass. The dominant violation is a minimum-pulse-width report on the divided clock register. The pixel-clock domain runs at 25 MHz (40 ns period), but the timing analyser treats the divider register as if it were producing a much higher-frequency clock; the combination of `posedge` and `negedge` from the alternating toggle makes the analyser see a 20 ns half-period. On real silicon the register is promoted to the global clock network and the duty cycle is exact, so the divider works as intended — but the analyser cannot see that, and reports failure.

The fix is to replace the divide-by-2 register with an `altera_pll` IP instance configured for 25 MHz, which the analyser models correctly. We have not done this yet because the Phase 1 board passes timing visually at lab temperature; the slack numbers will matter only when integration timing problems start mounting.

8.3 Frame timing

In the deployed (Phase 2) configuration, `top -n 1 -b` while `nml_game` is running shows CPU usage under 5%. The loop spends most of its time in `nanosleep` or polling `STATUS.SWAP_PENDING` inside `nml_commit_frame()`.

9 Challenges and Resolutions

9.1 Tile-map byte-enable RMW

The original tile-map storage was logic [7:0] `tilemap_ram [0:4799]` — one byte per word. The Linux driver's `nml_write_tile()` issues an ARM STRB for each tile update. The HPS-to-FPGA lightweight bridge sets `avs_byteenable` to one nibble and the Avalon slave's effective behaviour was a read-modify-write: read the 32-bit word, mask in the new byte, write back. With byte-organised storage, the slave only returned one byte from each read, so the bridge masked three zero bytes into the other lanes and silently clobbered them. The visible symptom was that only every fourth tile column showed new content; the rest stayed at reset value 0.

The fix is the packed logic [3:0][7:0] `tilemap_ram [0:1199]` storage plus a per-lane write in `hw/nml_gpu.sv:158-162`. Each lane updates independently, and reads return all four bytes, so the bridge's read-modify-write preserves the untouched bytes.

9.2 Single-cycle `eval_done` against multi-cycle `CLEAR_BUF`

`sprite_eval` finishes its 32-sprite walk in roughly 33 cycles and asserts `eval_done` for one cycle. `sprite_fetch` is in `CLEAR_BUF` for 640 cycles (one write per pixel) and only checks `eval_done` once, at the end of the buffer clear. If the eval pulse fires while fetch is still mid-clear, the pulse is gone by the time fetch samples it.

The first fix attempted was to edge-detect a transition on `active_mask`. That worked for the smoke-test scene because the active set changes between non-sprite and sprite rows. It silently failed for the game's much busier scenes: 15 of every 16 scanlines inside a 16-pixel sprite see the *same* active set, so no transition fires.

The current fix is a sustain latch: `eval_done_r` is set when the pulse arrives and cleared at the next `eval_strobe`. `sprite_fetch` samples `eval_done_r` and is guaranteed to see it set after the first eval of a line, regardless of when fetch happens to check (`hw/nml_gpu.sv:264-270`).

9.3 CDC on `ctrl_swap_req`

`CTRL.SWAP` is written from the 50 MHz Avalon clock and consumed in the 25 MHz pixel clock. The straight single-cycle pulse from the slave was 20 ns wide; a two-flop synchroniser at 25 MHz samples every 40 ns, so it could easily miss the pulse entirely.

The slave now widens the request: a 3-bit counter loaded with 7 on each SWAP write holds `ctrl_swap_req` high for seven 50 MHz cycles (140 ns), which is enough that a three-flop chain on the pixel side captures the rising edge deterministically.

9.4 Pixel-clock duty cycle

The divide-by-2 register works on silicon but fails the analyser's minimum-pulse-width check. We documented the failure mode and deferred the `altera_pll` swap until Phase 2 closes timing on its own merits. The divider's port names already match the IP's defaults, so the swap is a drop-in.

9.5 Active rendering bugs

In the current Phase 2 build, the VGA monitor shows the game across the full screen but with vertical stripes: the rendered pixels are correct in some columns and stale in others. The Game-Over screen also draws incorrectly — the centred text appears partially or not at all.

The stripe pattern is consistent with a partial mismatch between the byte-enable behaviour on writes and the byte-select behaviour on reads. The packed-byte tile-map fix in §9.1 addressed the gross clobbering, but a residual issue in the read-side byte select (`hw/nml_gpu.sv:169-173`) or in the way the slave decodes `avs_address` for the upper half of the tile-map region (`hw/avalon_slave_iface.sv:169-178`) is the most likely culprit. The decode there packs `{avs_address[13], avs_address[11:0]}` to handle the 0x1000–0x22BF range crossing bit 13; if a byte falls into the upper half (rows 25 and above), the lane the slave writes may not match the lane the compositor reads, producing the visible stripe.

The Game-Over visual is downstream of the same problem: the centred-text routine writes one byte at a time into the tile map, and the visible state depends on lane alignment.

Diagnosis of both is on the open-issues list. Both are intermittent on the smoke-test bitstream where the HPS does not write the tile map; this isolates the regression to the byte-write path under real load.

10 Conclusion and Open Work

The Phase 1 smoke-test bitstream works as designed on hardware: 640×480 @ 60 Hz, tile + sprite + palette + HUD pipeline, full VGA pin set. The Phase 2 HPS-integrated build boots Linux, exposes the peripheral on the lightweight bridge, runs the C game at 60 Hz, and reads SNES controller input via evdev. Player movement, firing, enemy spawning, wave progression, level-up, mortar timers, artillery, gas clouds, ammo drops, and the HUD numerics all function in the deployed configuration.

The two open items at the time of writing are the vertical-stripe rendering artefact on the HPS-integrated build (§9.5) and the Game-Over screen visual (§9.5), likely downstream of the first.

After those, the remaining work, in rough priority order:

- Swap `p11_25mhz` for `altera_p11` and close timing on the analyser.
- Wire `STATUS[15:8]` to an actual VSYNC counter (currently constant zero, `hw/nml_gpu.sv:231`).
- Sound: the WM8731 audio codec is on the board but not used.
- Real sprite art: the placeholder squares are functional but unattractive.
- Reduce the duplication between `hw/` and `nml_gpu_hw/` (sources are copied manually; symlinks or a Makefile rule would prevent drift).

References

1. `DESIGN.md` — Project design document, April 2026.
2. `SETUP.md` — Bring-up instructions for Phase 0, Phase 1, and Phase 2.
3. Terasic Inc., *DE1-SoC User Manual*, Revision 1.2.4.
4. Intel/Altera, *Cyclone V Device Handbook* (Volume 1).
5. Intel/Altera, *Avalon Interface Specifications*, MNL-AVABUSREF.

6. Analog Devices, *ADV7123 Triple High-Speed Video DAC Datasheet*.
7. VESA, *VGA 640×480 @ 60 Hz timing specification*.
8. Linux kernel evdev documentation: `Documentation/input/event-codes.rst`.
9. `lab3-reference/CSEE4840W-Lab3` — VGA ball lab providing the timing-generator skeleton.